

S&P 500 Market Direction

An End-to-End MLOps Pipeline (Proof of Concept)

Miguel Venâncio António Varela Maria Cruz Carlos Amorim

NOVA Information Management School (NOVA IMS), 2025/2026 • github.com/vehnie/sp500-mlops-pipeline

1 Data Choice, Objectives & Success Metrics

Why this data. We use daily OHLCV history of the S&P 500 index (Yahoo Finance, ticker ^GSPC, 2000–2026, 6,604 usable rows). The choice is deliberate for an *MLOps* project rather than a pure modelling one: the data is free, clean, reliably available, and updates every trading day, so it realistically exercises ingestion, scheduling, and drift monitoring. It is also a genuine time series, which forces correct, leak-free engineering (chronological splits, time-aware validation and monitoring), and it is a *low-signal* problem, exactly where solid operational tooling earns its value: small pipeline mistakes can masquerade as “performance”, so contracts and tracking are essential.

What we try to achieve. The task is binary classification: predict whether the next trading day closes higher than today, $y_t = \mathbb{I}[\text{Close}_{t+1} > \text{Close}_t]$. The real objective is to deliver a **production-grade ML lifecycle**: reproducible orchestration (Kedro), experiment tracking and a model registry (MLflow), data contracts (Great Expectations), drift monitoring (Evidently), explainability (SHAP), an offline feature store (Hopworks), a served REST API (FastAPI + Docker), and automated tests (pytest). The model itself is intentionally simple so the engineering around it is the deliverable.

Success metrics. We track success on two levels (Table 1). *Model metrics* are Accuracy, Precision, Recall, F1, and ROC-AUC, with **ROC-AUC as the primary discrimination metric**. *System (MLOps) metrics* define what “done” means operationally and are, for this PoC, the more important bar.

Table 1: Success criteria and final status.

Level	Criterion	Status
Model	ROC-AUC reported honestly on a leak-free chronological test set	✓ 0.499
Model	Champion beats challenger on the selection metrics (Acc/Recall/F1)	✓
System	One-command reproducible run (<code>kedro run</code>) from raw to predictions	✓
System	Data contracts pass at raw and feature layers	✓ 19/19, 27/27
System	Model tracked, versioned, and registered with lineage (MLflow)	✓
System	Engineered features published to a feature store with lineage	✓ Hopworks
System	Drift monitor produces an actionable report	✓
System	Champion served behind a REST API + container	✓
System	Automated test suite green	✓ 125 passed

2 System Architecture & Data Quality

The system is a directed acyclic graph of **Kedro** pipelines (Figure 1), each a set of nodes with declared inputs/outputs registered in a central data catalog, so every artifact is traceable to the node that produced it. A single `kedro run` executes the default end-to-end workflow: `data_quality` → `cleaning` → `feat_engineering` → `split` → `train` (+ challenger) → `predict`. Three further pipelines run on demand: `data_expectations` (contracts), `model_explainability` (SHAP), and `data_drift` (Evidently). Table 2 maps each MLOps concern to its tool. **Data quality** is enforced with Great Expectations at two checkpoints (raw

OHLCV and engineered features); both pass fully (19/19 and 27/27), so a future data refresh that breaks an assumption fails loudly and early.

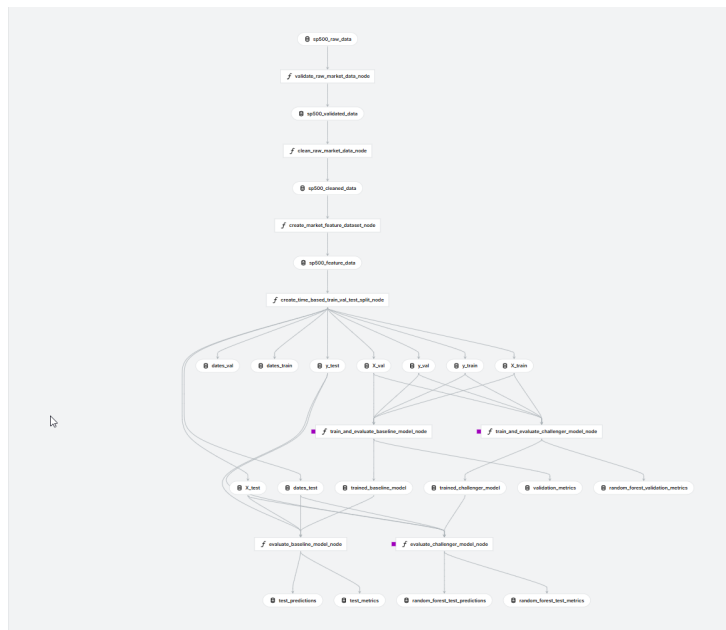


Figure 1: Kedro-Viz view of the end-to-end pipeline: raw data → validation → cleaning → feature engineering → chronological split → champion + challenger training → test evaluation. Every node and dataset is catalog-tracked and reproducible.

Table 2: MLOps concerns and the components implementing them.

Orchestration & catalog — Kedro	Drift monitoring — Evidently
Experiment tracking & registry — MLflow	Offline feature store — Hopsworks
Data & feature contracts — Great Expectations	Model serving (REST API) — FastAPI + Uvicorn
Explainability — SHAP (LinearExplainer)	Containerisation — Docker / Compose
Automated testing — pytest (125 tests)	Inspection dashboard — Streamlit (read-only)

3 Project Planning (Agile Sprints)

We organised the work as six one-week sprints with a Kanban board of pipeline-sized tasks. Each sprint produced a runnable increment: a new Kedro pipeline plus its tests and catalog entries were the “definition of done”, keeping the system end-to-end runnable at every stage so monitoring and serving could be added without touching the core training path.

Table 3: Sprint plan. Each sprint delivers one or more registered Kedro pipelines.

Sprint	Goal	Key deliverables
S0	Setup & exploration	Repo, Kedro scaffold, data_ingestion, EDA notebooks
S1	Data foundation	data_cleaning, data_feat_engineering, data_split (chronological 70/15/15), target definition
S2	Data quality	data_quality + data_expectations (Great Expectations contracts)
S3	Modelling & tracking	model_train (LR champion), model_train_challenger (RF), MLflow tracking + registry
S4	Evaluation & explainability	model_predict(_challenger), reporting plots, model_explainability (SHAP)
S5	Feature store, monitoring & serving	feature_store (Hopsworks), data_drift (Evidently), FastAPI + Docker, Streamlit, pytest, docs/report

4 Results: Data Exploration & Modelling

Exploration. The index level is strongly trending and *non-stationary*, while daily returns are stationary around zero with clustered volatility (Figure 2); the target is mildly skewed toward up-days ($\approx 53\%$). From price and volume we engineer eight leak-free features (returns, sma_10/20, sma_ratio_10, rsi_14, volatility_10, volume_change), kept small so every feature stays inspectable. The stationarity contrast directly explains the drift results in Section 5.

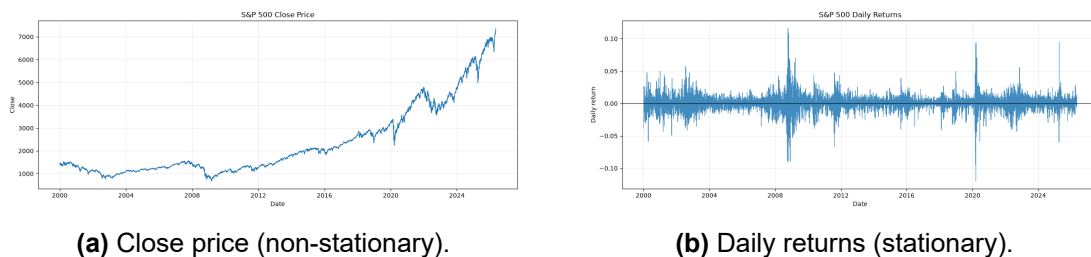


Figure 2: Exploratory analysis. Trending levels vs. stationary returns motivate the feature-drift findings below.

Modelling & explainability. A **champion** (StandardScaler + Logistic Regression) is compared with a **challenger** (Random Forest, 300 trees, depth 8, regularised), both under the identical chronological split with MLflow tracking. On the 992-row future test block (Table 4) the champion leads on Accuracy/Recall/F1 and is chosen, but **both ROC-AUC values are ≈ 0.50** (Figure 3a): neither model ranks up-days above down-days better than chance. The champion's high recall (0.922) merely reflects predicting "up" most days, exploiting the class prior. SHAP on the champion (Figure 3b) ranks recent risk (volatility_10) and distance from trend (sma_ratio_10) highest, but all attributions are *small*. This is the honest, expected outcome for price-only daily forecasting; its value is that the leak-free split, passing contracts, and small SHAP magnitudes together confirm the weak signal is *real*, not a leak.

Table 4: Test-set performance. Champion selected on Accuracy/Recall/F1; both show near-random ROC-AUC.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (champion)	0.536	0.542	0.922	0.683	0.499
Random Forest (challenger)	0.503	0.547	0.473	0.507	0.502

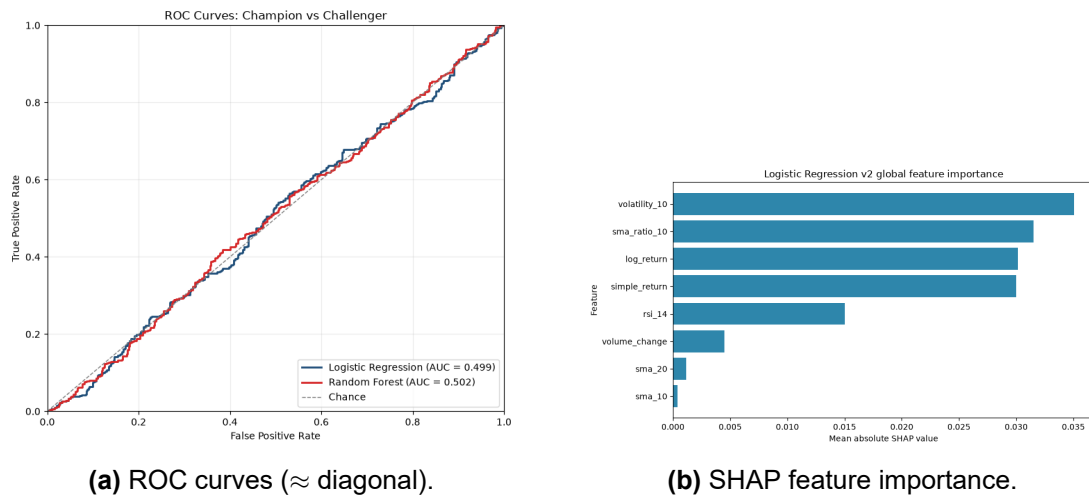


Figure 3: (a) Both ROC curves hug the chance diagonal ($AUC \approx 0.50$). (b) SHAP ranks `volatility_10` and `sma_ratio_10` highest, but magnitudes are small, consistent with a weak signal.

5 MLOps in Practice: Tracking, Monitoring, Feature Store & Serving

The operational layer is where this project does its real work; Figure 4 shows it running end to end.

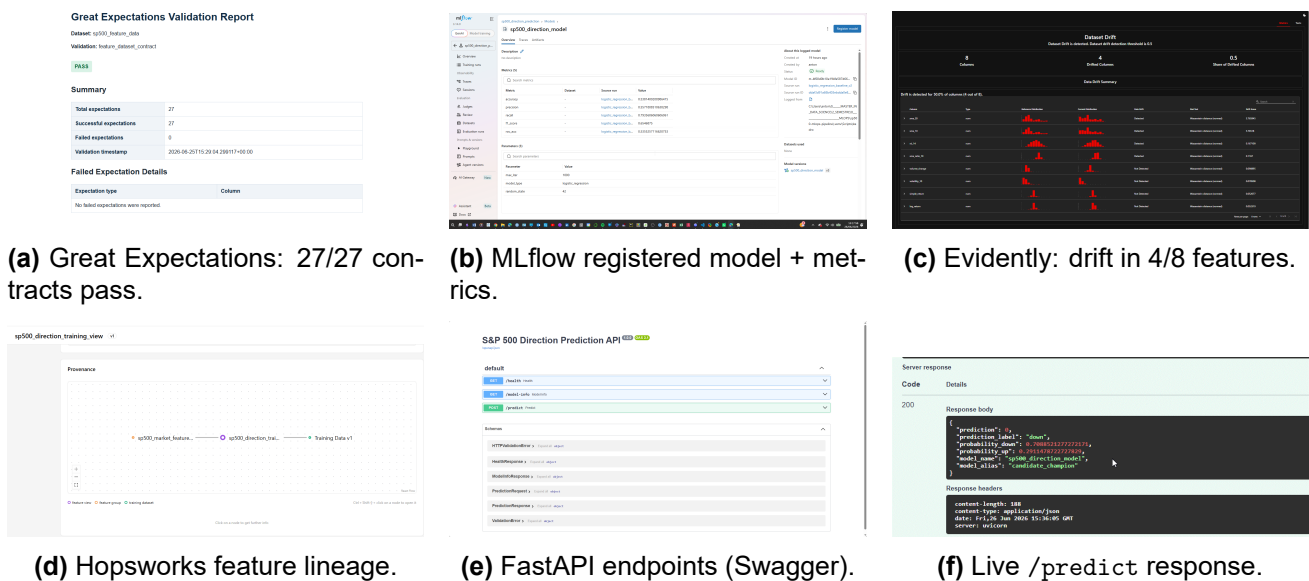


Figure 4: The MLOps lifecycle in practice. (a) Data contracts gate the data before training. (b) Every run is tracked in MLflow; the champion is registered as `sp500_direction_model` (alias `candidate_champion`). (c) Evidently flags dataset-level drift (4/8 features). (d) Hopworks stores engineered features with lineage from Feature Group to training view. (e–f) The champion is served through a FastAPI REST API returning a class and probabilities.

Tracking & registry. Every run logs parameters, metrics, tags (`split_strategy = chronological_70_15_15`), the feature list, the model signature, and the serialised artifact to MLflow. The champion is registered as `sp500_direction_model` and loaded for serving by the stable alias `candidate_champion`, so a newly promoted model is picked up without code changes.

Monitoring. Evidently compares the oldest 70% (reference) against the newest 30% (current) with a normalised Wasserstein distance (threshold 0.1) and flags dataset-level drift: the non-stationary moving averages `sma_10/sma_20` drift heavily (scores > 6) while the stationary

return features do not, pinpointing exactly which features a production system should reformulate.

Feature store & serving. Engineered features are published to a Hopsworks offline feature store (Feature Group `sp500_market_features v1` → training Feature View `sp500_direction_training_view v1`) for reuse and lineage. The FastAPI service exposes `/health`, `/model-info`, and `/predict` (Pydantic-validated eight-feature input → class + probability) and is packaged as the Docker image `sp500-direction-api`. Reproducibility is structural: catalog-declared datasets, versioned `conf/` parameters, fixed random states, and a 125-test pytest suite plus the data contracts give two independent safety nets.

6 From Proof of Concept to Production

This PoC runs locally (Pandas in-memory, local MLflow with a SQLite backend, `yfinance` ingestion), but the technology choices are production-aligned, so deployment is incremental rather than a rewrite. **Advantages of the stack:** Kedro maps cleanly onto an orchestrator (Airflow/Kubeflow); MLflow already provides lineage, versioning, and a registry the API loads by alias; Great Expectations and Evidently make data quality and drift CI-friendly; Hopsworks gives feature reuse and train/serve parity; FastAPI + Docker yield a portable, scalable service; pytest plus contracts give two safety nets. Table 5 lists the main limitations with effort-scoped mitigations.

Table 5: Production risks and proposed mitigations (effort as additional sprints).

Risk / limitation	Proposed mitigation	Effort
Pandas is single-node; will not scale to large/intraday data	Port feature/cleaning nodes to Spark (Kedro keeps the same node interface)	+3 wk
Local MLflow (SQLite + file store) is not multi-user or durable	Managed tracking server + cloud artifact store (S3/Postgres backend)	+1 wk
Hopsworks feature store is offline-only (no low-latency serving)	Enable Hopsworks online feature serving for real-time inference parity	+2 wk
<code>yfinance</code> is an unofficial, rate-limited source	Scheduled ingestion via a licensed vendor + retries/back-off	+1 wk
Drift is detected but retraining is manual	CI/CD with drift-triggered automated retraining & promotion gates	+2 wk
Weak signal (ROC-AUC ≈ 0.50)	Add alternative data (macro, sentiment, cross-asset), calibrate, walk-forward backtest	+3 wk
API has no auth / observability	Add auth, rate limiting, logging, metrics (Prometheus/Grafana)	+1 wk

7 Packages and Versions

Developed on **Python 3.11.9**. Core dependencies (pinned versions used for the final run):

```

• kedro==1.4.0
• yfinance==1.4.1
• numpy==2.4.6
• kedro-datasets==9.4.0
• pandas==2.3.3
• scikit-learn==1.9.0

```

-
- | | | |
|----------------------|------------------------------|------------------------|
| • matplotlib==3.11.0 | • uvicorn==0.49.0 | • streamlit==1.58.0 |
| • mlflow==3.14.0 | • pydantic==2.13.4 | • python-dotenv==1.2.2 |
| • shap==0.51.0 | • great_expectations==1.18.1 | • hopsworks |
| • fastapi==0.138.0 | • evidently==0.7.21 | • pytest==9.1.1 |
-

References

- [1] Yahoo Finance. S&P 500 index (^GSPC) historical data. <https://finance.yahoo.com/quote/%5EGSPC/chart/>